Ezlo Plugin Development: Quickstart

Ezlo Innovations

September 2022

1 Introduction

This document will be a short introduction to plugin development in the ezlogic plugin platform, providing the basic information needed to quick-start development.

2 Ezlogic

The end-user interface for the Ezlo hub is the ezlogic website. This is the website you'll use to upload, install, and test your plugin. In order to access it, you'll need an Ezlo/Vera account. So the first step, is to create an account, and log-in.

After that, you'll be greeted to the dashboard, and a sidebar on the left. 3 Tabs in specific are noteworthy:

- **Automation** → **MeshBots**
  This is where the end-user will use the capabilities of your plugin to create and manage custom automations.
- **Plugins** → **Edge Plugins** This is where you can upload, install, configure, and remove plugins from your hub. Two methods of installing are available: Uploading a local file, and downloading from the marketplace. During development, you will be uploading from your local machine. When the plugin is finished, it will be uploaded to the marketplace, where the end-user will install it from.
- **Settings** → **Devices** This is where devices are displayed. These can be hubs, physical devices connected, or virtual devices created by plugins. From here, they can be manually viewed, reconfigured, and removed by the user.

For the sake of brevity, we will from now on call these "MeshBots tab", "plugins tab", and "devices tab".

3 Virtual Hub

If you have no physical Ezlo Hub, the easiest way to start development is with SoftHub. This is a virtual hub, which emulates a physical Ezlo Hub on your computer.

3.1 With Z-Wave Hardware

If you have some Z-Wave hardware, you can just use the instructions in the community post.

3.2 Without Z-Wave Hardware

If not, then a simple docker-compose should do the job[1], by using the vhub docker-compose folder provided[2] (i.e. running `sudo docker-compose up` inside the folder).

After that, with the vhub running, use the provision tool to link the hub to your account. The exact command is `./provision.linux.amd64 provision --username <ACCOUNT USERNAME> --password <ACCOUNT PASSWORD > --description <SOME DESCTIPTION>`[3].

4 Minimal file structure

Every plugin uploaded is a `tar.gz` archive, with a `config.json` file, `interface.json` file, and a `scripts` folder.

The minimal file structure for a plugin is the following:

```
plugin_name.tar.gz/
  config.json
  interface.json
  scripts/
    startup.lua
    teardown.lua
```

We'll now go into details for each of these files.

4.1 `config.json`

This file is where you define the internal settings for the plugin. Here's a minimal `config.json`:

[1] These instructions are, currently, linux only

[2] Two versions were provided. An older but stable one, and a bleeding-edge but unstable one

[3] You can run `./provision.linux.amd64 --help` for all the options available.

```
{
  "id": "ezlo.minimal",
  "version": "1.0.0",
  "meta": {
    "name": {
      "text": "Minimal Test Plugin"
    },
    "description": {
      "text": "A plugin with the bare minimal configs, and no
          functionality, used purely for exposition."
    },
    "author": {
      "text": "Ezlo Innovation"
    },
    "type": "node",
    "language": "lua",
    "placement": {
        "static": true,
        "custom": true
    }
  },
  "type": "gateway",
  "dependencies": {
    "firmware": "2.0",
    "addons": [
      {
        "id": "lua",
        "version": "1.0"
      }
    ]
  },
  "permissions": [
    "core",
    "logging",
    "storage"
  ],
  "executionPolicy": "restoreLastScriptState",
  "startup": "scripts/startup",
  "teardown": "scripts/teardown",
  "gateway": {
    "name": "ezlo.minimal",
    "label": "Minimal test plugin",
    "setItemValueCommand": "HUB:ezlo.minimal/scripts/
```

```
        set_item_value",
43      "setSettingValueCommand": "HUB:ezlo.minimal/scripts/
        set_setting_value",
44      "forceRemoveDeviceCommand": "HUB:ezlo.minimal/scripts/
        delete_device"
45    }
46  }
```

Most of these are simply sane defaults. Here are the most noteworthy ones:

- `id`: This is a unique identifier for the plugin. It consists of a prefix and a name. If you're developing a plugin for Ezlo Innovations, the prefix should be `ezlo`. If not, you can grab a prefix for yourself in the 'Plugins → Plugin Settings' tab on ezlogic. It'll be used throughout the plugin's code, and should be unique for your plugin.
- `version`: This is the version of the plugin. It is mostly decided by you, but it should be $\geq$ 1.0.0
- `meta`: Holds some data about the plugin, such as name, description, and author. These will be displayed in the UI to the user.
- `permissions`: This specifies the modules the plugin will access. You will *not* be able to `require` a module without specifying it in here.
- `startup`: This file will execute when first installing the plugin. It should contain some initial setup.
- `teardown`: This file will execute when uninstalling or upgrading the plugin. It should contain some cleaup code.
- Gateway Commands: The scripts specified here are executed when certain actions or events happen, such as an item or setting being changed, or a device being removed. The full list of available commands are specified in the API Docs.

4.2 `interface.json`

In this script, you specify details about the user interface. Here's an example:

```
1  {
2    "configuration":{
3      "type":"static",
4      "script":"ezlo.minimal/scripts/configure_new",
5      "inputs":[
```

```
      {
        "name":"String",
        "description":"String input",
        "field":"input",
        "required"true,
        "type":"string"
      },
      {
        "name":"Integer",
        "description":"Integer input",
        "field":"input",
        "required":false,
        "type":"int"
      }
    ]
  }
}
```

- `configuration.script`: This file will execute when the user creates a new configuration. When it executes, it'll receive all of the inputs specified, which will be present as fields in the `"..."` lua variable.
- `configuration.inputs`: A list of all of the inputs the configuration should take. This includes the UI name and description, type, if it is required or not, and the field that will be used when the configuration script is called.

4.3 `scripts` folder

This folder will contain the code for your plugin. It may contain any amount of (nested) folders and lua files to be executed. Any file can be accessed/executed using the `"HUB:<PLUGIN_ID>/scripts/<FILEPATH>"` idiom by any of the usual lua means (i.e. `require`, `loadfile`, etc).

For example, `require("HUB:ezlo.minimal/scripts/configure_new")` will import the file `configure_new.lua` that is inside the scripts folder.

5 Uploading

To execute your plugin, it must be uploaded to the ezlogic platform (in a `tar.gz` format, as specified above).

Go to the plugins tab, click on "Upload New Plugin", and select the archive. The plugin will appear in the list of plugins. Press the plugin's

"Manage Installations" icon, install, and configure it. If everything went right, your plugin should now be functioning.

A minimal example plugin will be provided with this document. Feel free to install it, explore its codebase, and use it as a starting point for your own plugin.

6 API Basics

Some pieces of the API are so central, that you'll almost certainly use them during development. In this section, we'll give them some light. Note that we'll only discuss *some* parts of *some* of the modules. Your ultimate guide should be the official MiOS API documentation, specially the modules section. When in doubt, refer to it.

All of the code snippets present is this section will also be present in the minimal example plugin provided. So, feel free to explore there as well.

6.1 Storage

There are 3 levels of storage: Local, Global, and Persistent.

- Local: Any `local` variable will only be accessible to that script, and to its appropriate scope.
- Global: Any global variable will be accessible to *all* of the plugin's scripts. This is important, as it allows different scripts to communicate/share data.
- Persistent: Any variable stored with the `storage` module, will be stored persistentently. These are accessible by any script, *and* throughout all executions of the plugin. They will survive shutdowns, reboots, power outages, etc.

```lua
-- Importing storage module.
local storage = require("storage")

-- Local variables.
local num_0 = 1
local str_0 = "abc"
local tab_0 = {1, 3, 5}

-- Global variables.
num_1 = 2
str_1 = "def"
_G.tab_1 = {2, 4, 6} -- Alternative way to express it
```

```lua

-- Persistent variables (storing).
storage.set_number("num_2", 3)
storage.set_string("str_2", "ghi")
storage.set_table("tab_2", {3, 5, 7})
-- Persistent variables (retrieving).
local num_2 = storage.get_number("num_2")
local str_2 = storage.get_string("str_2")
local tab_2 = storage.get_table("tab_2")
```

Higher levels of storage are less efficient, so try use the least storage level you need. If there's no reason for something to be persistent, make it global. If there's no reason for something to be global, make it local.

6.2 Logging

You can log data using the `logging` module. There are different levels of logging, such as `info`, `debug`, and `error`. The complete list is presented in the documentation.

These different levels are useful for different purposes. For example, `debug` logs will not be generated for end users, so they may be more extensively used without affecting the end-user's storage space.

```lua
-- Importing logging module.
local logging = require("logging")

-- Examples of logging.
logging.info("3 devices added.")
logging.error("Could not create connection")
local t = {1, 'abc'}
logging.debug(t)
```

So where do the logs go? To '`/var/log/firmware/ha-luad.log`'[4] inside the hub. So, to access them, you need to get into the hub:

- If you have a physical hub, you can to ssh to it (i.e. `ssh mypc@106.000.0.1`).
- If you have a virtual hub, you can docker exec bash to it (i.e. `sudo docker exec -it vhub_beta bash`).

Once you're in, you can vizulize the logs in whichever way you wish. For example, with `tail`: `tail -f -n 50 /var/log/firmware/ha-luad.log`.

[4] It's worth noting that there are lot more log files inside `firmware` folder. `ha-luad.log` is simply the one generated by plugins. These other log files contain useful information about different aspects of the hub.

```
        **** luad Restarted at Fri Sep 23 15:20:45 UTC 2022

        2022-09-23 15:20:45 INFO  : Logs folder was changed: //
            var/log/firmware
        2022-09-23 15:20:45 INFO  : addon.lua: - [2022-08-10T20
            :49:14+0000]
        2022-09-23 15:20:45 INFO  : addon.lua: Spread:
            connected to "4803" with private group "#addon.lua#
            localhost"
```

6.3 Asynchronous Execution

Development in the API is based upon asynchronous execution. This means that individual scripts can be called to be executed by asynchronous events, such as a network input, http response, or a specific timer set by you. These are handled by either subscribing to an event, or manually scheduling the execution of a script. Let's see an example of each.

6.3.1 `network` module

The network package handles transport-layer connections, such as UDP and TCP. When using a persistent TCP connection, you can send or receive data from these connections. To be sure that your handler script will execute to process that data, you may `subscribe` the script to that connection:

```
local network = require("network")

-- The file 'network_handler.lua' will now be called on every
    network event.
network.subscribe("HUB:ezlo.minimal/scripts/network_handler")

-- We then attempt a connection.
network.connect{
        ip = '127.0.0.1',
        port = '5000',
        connection_type = 'TCP',
}
-- This connection attempt will queue the execution of '
    network_handler.lua'.
```

Note that the queued script will get some data about the event that triggered it. This data can be accessed through the scripts' `...` arguments:

```lua
local logging = require("logging")

-- This is getting the arguments, which will include the event's data.
local args = ...

-- Logging event's data.
logging.debug(args)
```

6.3.2 `timer` module

The timer module is how your plugin can manually schedule a script to execute.

```lua
-- Importing timer module.
local timer = require("timer")

-- Immediately queues 'script_0.lua' to be executed.
timer.set_timeout(0, "HUB:ezlo.minimal/scripts/script_0")

-- Queues 'script_1.lua' to be executed in 5 seconds.
timer.set_timeout(5000, "HUB:ezlo.minimal/scripts/script_1")

-- Queues 'script_2.lua' to be executed every 10 seconds.
timer.set_interval(10000, "HUB:ezlo.minimal/scripts/script_2")
```

6.3.3 Extra notes about asynchronous execution

One important point to emphasize is that the asynchronous system operates in a queue fashion, with new scripts added to the end of the queue, to be executed one after the other. There are two major consequences of this:

- The system is *not* multithreaded. So despite its asynchronous nature, data races are not a concern, as one script must finish before the others can execute.
- If one script falls into an inifinite (or arbitrarily long) loop, the plugin halts, since all other queued scripts must wait for the current one to finish.

6.4 Devices

Devices are a representation of an entity that acts or can be acted upon by automation. It may be a 'model' for a physical device, such as a light

bulb, switch, or sensor. It might also be a purely virtual one, like a client that communicates to a server. Devices can be created, accessed, and modified using the core module:

```
local core = require("core")

-- Creating new device.
local device_id = core.add_device{
        type = 'device',
        device_type_id = 'api_communication',
        category = 'generic_io',
        subcategory = 'generic_io',
        battery_powered = false,
        gateway_id = core.get_gateway().id,
        -- This name will be displayed in the UI.
        name = "API Communication Client",
}

-- ...
```